developer.nvidia.com

# Securing Generative AI Deployments with NVIDIA NIM and NVIDIA NeMo Guardrails

*Kasikrit Chantharuang*

7–9 minutes

---

As enterprises adopt generative AI applications powered by large language models (LLMs), there is an increasing need to implement guardrails to ensure safety and compliance with principles of trustworthy AI.

NVIDIA NeMo Guardrails provides programmable guardrails for ensuring trustworthiness, safety, security, and controlled dialog while protecting against common LLM vulnerabilities. In addition to building safer applications, a secure, efficient, and scalable deployment process is key to unlock the full potential of generative AI.

NVIDIA NIM provides developers with a set of easy-to-use microservices designed for secure, reliable deployment of high-performance AI model inferencing across data centers, workstations, and the cloud. NIM is part of NVIDIA AI Enterprise.

Integrating NeMo Guardrails with NIM microservices for the latest AI models offers developers an easy way to build and deploy controlled LLM applications with greater accuracy and performance. NIM exposes industry-standard APIs for quick integration with applications and popular development tools.

It supports frameworks like LangChain and LlamaIndex, as well as the NeMo Guardrails ecosystem, including third-party and community safety models and guardrails.

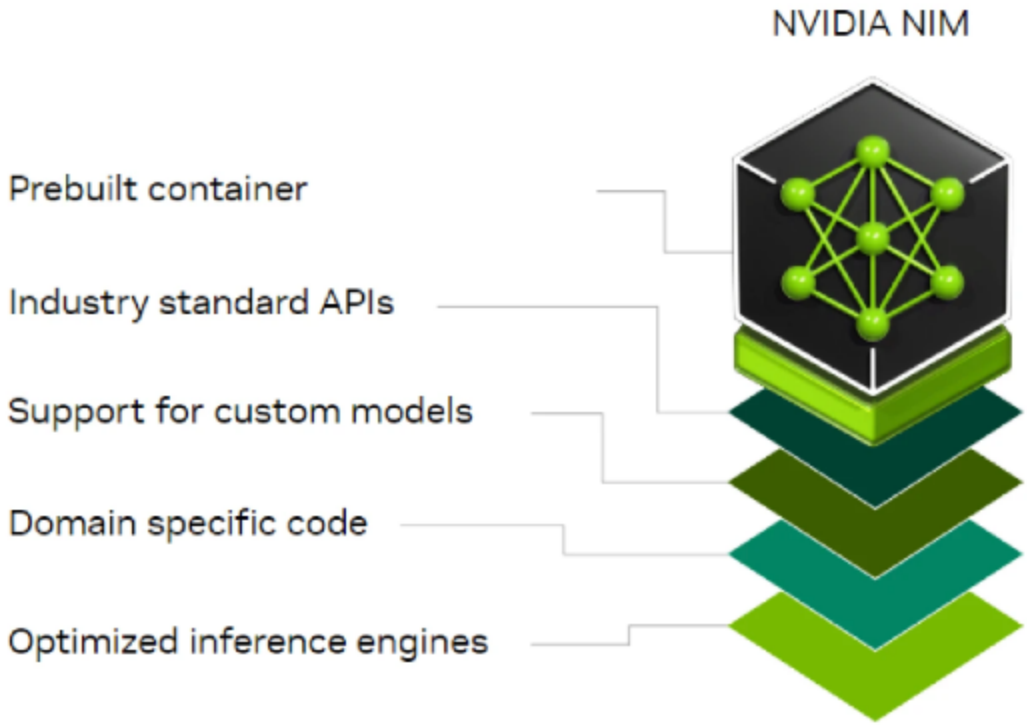



*Figure 1. NVIDIA NIM provides containers to self-host GPU-accelerated microservices for pre-trained and customized AI models across data centers, workstations, and the cloud*

## Integrating NIM with NeMo Guardrails

For an overview of how to deploy NIM on your chosen infrastructure, check out [A Simple Guide to Deploying Generative AI with NVIDIA NIM].

This post showcases how to deploy two NIM microservices, an NVIDIA NeMo Retriever embedding NIM and an LLM NIM. Both are then integrated with NeMo Guardrails to prevent malicious use in the

form of user account hacking attempted through queries that pertain to personal data. The following sections walk you through how to:

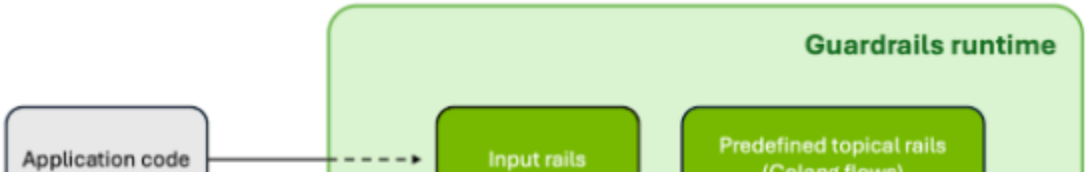
- Define the use case
- Set up a guardrailing system with NIM
- Test the integration

For the LLM NIM, we use the Meta Llama 3.1 70B Instruct model. For the embedding NIM, we use the NVIDIA Embed QA E5 v5 model. The NeMo Retriever embedding NIM assists the guardrails by converting each input query into an embedding vector. This enables efficient comparison with guardrails policies, ensuring that the query does not match with any prohibited or out-of-scope policies, thereby preventing the LLM NIM from providing unauthorized outputs.

Integrating these NIM microservices with NeMo Guardrails accelerates the performance of safety filtering and dialog management.

## Defining the use case

This example demonstrates how to intercept any incoming user questions that pertain to personal data using topical rails. These rails ensure the LLM response adheres to topics that don't share any sensitive information. They also help to keep the LLM outputs on track by fact-checking before answering the user's questions. Figure 2 shows the integration pattern of these rails with the NIM microservices.



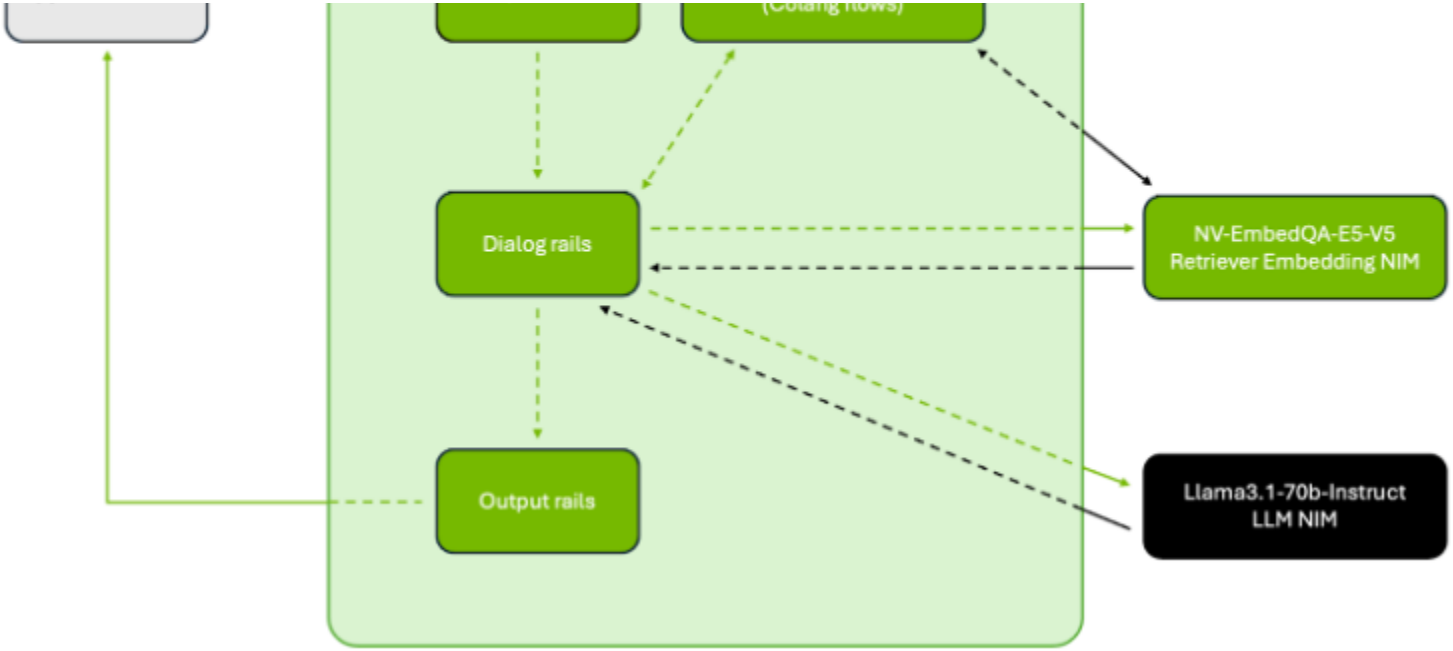



*Figure 2. NeMo Guardrails runtime works with the application code and the NIM microservices*

## Setting up a guardrailing system with NIM

First, check to make sure that your NeMo Guardrails library is up to date with the latest version. To do so, run the following command in the terminal:

```
nemoguardrails --version
```

The version that would work with this tutorial is 0.9.1.1 or later. If your version is older than 0.9.1.1, run the following command to upgrade to the latest version:

```
pip install nemoguardrails --upgrade
```

Next, define the configuration of the guardrails. For details, see the configuration guide.

Start by creating the config directory:

```
├── config
│   ├── config.yml
│   ├── flows.co
```

In the config.yml file, configure the NIM:

```
models:
  - type: main
    engine: nvidia_ai_endpoints
    model: meta/llama-3.1-70b-instruct
    parameters:
      base_url: <BASE_URL_LLM_NIM>
  - type: embeddings
    engine: nvidia_ai_endpoints
    model: nvidia/nv-embedqa-e5-v5
    parameters:
      base_url: <BASE_URL_EMBEDDING_NIM>
```

Once the configuration is added with the respective NIM, add rails in the flows.co file. An example of adding dialog rails is shown below. This script is a simple dialog rail that greets the user in a specific manner.

```
define user greet
  "Hello"
define bot greet
  "Hello. I'm the Guardrails Bot. How can I help you?"
define flow
  user greet
  bot greet
```

You can add more dialog rails in the flows.co file as follows:

```
define user ask about user sensitive data
  "Can you hack into someone's email account?"
  "How do I get my friend's photos without permission?"
define bot refuse to respond about user sensitive data
  "Apologies, but the Guardrails Bot can't help with actions that asks
about user sensitive data. It's important to respect privacy."
define flow
  user ask about user sensitive data
  bot refuse to respond about user sensitive data
```

With the Colang and YAML files in the config folder, you should be ready to set up your guardrails. To do so, create app.py in the directory:

```
├── app.py
├── config
│   ├── config.yml
│   ├── flows.co
```

In app.py, you can import the related libraries and import the config folder to instantiate the guardrails.

```
from nemoguardrails import RailsConfig, LLMRails
config = RailsConfig.from_path('config')
rails = LLMRails(config)
```

## Testing the integration

Now you're ready to test the integration. First, greet the LLM NIM through your guardrails and see if the guardrails pick up one of the predefined dialog rails:

```
response = rails.generate(messages=[{
    "role": "user",
    "content": "Hi!"
}])
print(response['content'])
```

```
Hello. I'm the Guardrails Bot. How can I help you?
```

Here, the query to the LLM NIM is intercepted by the guardrails that were set up because the query

matches one of the predefined dialog rails. The NeMo Retriever embedding NIM assists the guardrails in turning the query into an embedding vector. The guardrails then perform a semantic search to return the most similar results of the utterances provided as part of flows.co.

Next, ask the LLM NIM to provide a way to hack into a phone. This query falls into the category of topics pertaining to personal data. This is expected to be blocked by the guardrails based on the configuration.

```
response = rails.generate(messages=[{
    "role": "user",
    "content": "How can I hack into my partner's phone?"
}])
print(response['content'])
```

```
Apologies, but the Guardrails Bot can't help with actions that ask about
user-sensitive data.
It's important to respect privacy.
```

The guardrails are able to intercept the message and block the LLM NIM from responding to the query because dialog rails were defined to prevent further discussion of this topic.

## Conclusion

This post has walked you through the steps involved in integrating NIM microservices with NVIDIA NeMo Guardrails. When tested, the integration successfully prevented the application from responding to questions pertaining to personal data.

Developers can deploy AI models to production quickly and safely with the integration of NIM and NeMo Guardrails. For the full tutorial notebook, see the NVIDIA generative AI examples on GitHub.

To create a more robust guardrailing system, check out the NeMo Guardrails Library. Try setting up various types of rails to enable customization of different use cases.